

SMSM — Architecture Back-end

Référence des couches, objets et mappers • Spring Boot 3 / Java 21

1. Vue d'ensemble de l'architecture en couches

Pour chaque feature métier (Equipe, Joueur, Match, Article...), on crée systématiquement les mêmes objets dans les mêmes packages. Cette cohérence rend le code prévisible et maintenable.

Couche	Objet	Package	Rôle en une phrase
BDD	EquipeEntity	entity/	Miroir exact de la table SQL. Annotations JPA. Jamais exposé à l'extérieur.
DAO	EquipeRepository	repository/	Porte d'entrée vers la BDD. Hérite de JpaRepository. Génère les requêtes SQL.
Mapping	EquipeEntityMapper	mapper/	Convertit Entity ↔ Domain. Aucune logique métier.
Métier	Equipe	domain/	Objet métier pur. Indépendant de JPA et du front. Manipulé par le Service.
Mapping	EquipeDtoMapper	mapper/	Convertit Domain ↔ DTO. Aucune logique métier.
DTO	EquipeDto	dto/	Objet exposé à l'API. Contrôle ce que le front reçoit ou envoie.
Service	EquipeService	service/	Cerveau de l'application. Logique métier, validations, orchestration.
HTTP	EquipeController	controller/	Expose les endpoints REST. Appelle le Service. Ne parle jamais au Repository.

2. Flux complet d'une requête GET /api/equipes

```
Requête HTTP → Controller
↓ appelle
Service
↓ appelle
Repository → PostgreSQL → List
↑
EquipeEntityMapper → List (domain)
↑
Service retourne List
↑
EquipeDtoMapper → List
↑
Controller → HTTP 200 + JSON
```

Règle absolue : chaque couche ne parle qu'à sa voisine directe. Le Controller ne touche jamais le Repository. Le Repository ne connaît pas les DTO.

3. Détail de chaque objet

EquipeEntity — couche BDD

Annotation clé	@Entity, @Table, @Id, @GeneratedValue, @Column
Dépendance	Jakarta Persistence (JPA)
Contient	Les champs exacts de la table SQL
Ne contient pas	Logique métier, calculs, règles de validation
Lombok	@Getter @Setter @NoArgsConstructor @AllArgsConstructor

```
@Entity
@Table(name = "equipe")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class EquipeEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "nom", nullable = false)
    private String nom;
    private String categorie;
    private String description;
}
```

EquipeRepository — couche DAO

Annotation clé	@Repository (implicite via JpaRepository)
Hérite de	JpaRepository
Génère automatiquement	findAll(), findById(), save(), deleteById()...
Tu peux ajouter	findByNom(String nom) → Spring génère le SQL
Ne contient pas	Logique métier

```
@Repository
public interface EquipeRepository
    extends JpaRepository {

    // Spring génère : SELECT * FROM equipe WHERE nom = ?
    Optional findByNom(String nom);
}
```

Equipe — objet métier (domain)

Annotation clé	Aucune annotation Spring ou JPA
Rôle	Représentation pure de l'équipe dans la logique applicative
Indépendant de	JPA, HTTP, JSON — juste du Java pur
Manipulé par	Le Service uniquement
Lombok	@Getter @Setter @NoArgsConstructor @AllArgsConstructor

```
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Equipe {
    private Long id;
    private String nom;
    private String categorie;
    private String description;
}
```

EquipeDto — couche exposition

Annotation clé	Aucune (ou @JsonProperty si besoin de renommer)
Rôle	Ce que le front reçoit ou envoie — tu choisis les champs exposés
Peut différer de	L'Entity (ex: pas d'id en création, champs renommés...)
Lombok	@Getter @Setter @NoArgsConstructor @AllArgsConstructor

```
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class EquipeDto {
    private Long id;
    private String nom;
    private String categorie;
    private String description;
}
```

EquipeEntityMapper — Entity ↔ Domain

Annotation clé	@Component (bean Spring, injectable)
Rôle	Convertit EquipeEntity → Equipe et Equipe → EquipeEntity
Ne contient	Aucune logique métier, seulement du mapping de champs
Utilisé par	Le Service

```
@Component
public class EquipeEntityMapper {

    public Equipe toDomain(EquipeEntity entity) {
        return new Equipe(entity.getId(), entity.getNom(),
            entity.getCategorie(), entity.getDescription());
    }

    public EquipeEntity toEntity(Equipe domain) {
        return new EquipeEntity(domain.getId(), domain.getNom(),
            domain.getCategorie(), domain.getDescription());
    }
}
```

EquipeDtoMapper — Domain ↔ DTO

Annotation clé	@Component
Rôle	Convertit Equipe → EquipeDto et EquipeDto → Equipe
Utilisé par	Le Controller

```
@Component
public class EquipeDtoMapper {

    public EquipeDto toDto(Equipe domain) {
        return new EquipeDto(domain.getId(), domain.getNom(),
            domain.getCategorie(), domain.getDescription());
    }

    public Equipe toDomain(EquipeDto dto) {
        return new Equipe(dto.getId(), dto.getNom(),
            dto.getCategorie(), dto.getDescription());
    }
}
```

EquipeService — couche métier

Annotation clé	@Service
Rôle	Toute la logique métier : validation, règles, orchestration
Injecte	EquipeRepository + EquipeEntityManager
Retourne	Des objets Equipe (domain), jamais des Entity ni des DTO
Exemple de règle	Vérifier qu'une équipe au même nom n'existe pas avant création

```
@Service
@RequiredArgsConstructor // Lombok : injecte via constructeur
public class EquipeService {

    private final EquipeRepository repository;
    private final EquipeEntityManager mapper;

    public List findAll() {
        return repository.findAll()
            .stream()
            .map(mapper::toDomain)
            .toList();
    }
}
```

EquipeController — couche HTTP

Annotation clé	@RestController + @RequestMapping("/api/equipes")
Rôle	Reçoit les requêtes HTTP, appelle le Service, retourne du JSON
Injecte	EquipeService + EquipeDtoMapper
Ne contient pas	Logique métier — délègue tout au Service
Verbes HTTP	GET (lecture), POST (création), PUT (modification), DELETE (suppression)

```
@RestController
@RequestMapping("/api/equipes")
@RequiredArgsConstructor
public class EquipeController {

    private final EquipeService service;
    private final EquipeDtoMapper mapper;

    @GetMapping
    public List findAll() {
        return service.findAll()
            .stream()
            .map(mapper::toDto)
            .toList();
    }
}
```

4. Structure des packages

```
com.smsm.backend/  
■■■ BackendApplication.java  
■■■ controller/  
■ ■■■ EquipeController.java  
■■■ service/  
■ ■■■ EquipeService.java  
■■■ repository/  
■ ■■■ EquipeRepository.java  
■■■ entity/  
■ ■■■ EquipeEntity.java  
■■■ domain/  
■ ■■■ Equipe.java  
■■■ dto/  
■ ■■■ EquipeDto.java  
■■■ mapper/  
■■■ EquipeEntityMapper.java  
■■■ EquipeDtoMapper.java
```

5. Règles d'or à respecter

Objet	Règle
Controller	N'appelle jamais le Repository directement. Passe toujours par le Service.
Service	Ne retourne jamais une Entity ou un DTO. Retourne toujours un objet Domain.
Repository	Ne contient aucune logique métier. Seulement des requêtes de données.
Mapper	Ne contient aucune logique métier. Seulement des conversions de champs.
Entity	N'est jamais exposée à l'extérieur de l'application (pas de @RestController qui retourne une Entity).
DTO	N'est jamais utilisé dans le Service. Le Service parle Domain, pas DTO.

Ce document est un modèle — reproduire ce pattern pour chaque feature : Joueur, Match, Article, Utilisateur.